Sprawozdanie z Laboratorium nr 1

Przedmiot: Integracja Systemów Informatycznych

Temat: Git, GitHub i przygotowanie środowiska Django

Data: 2026-03-17

Student: Aleksandr Vasiljev

---

1. Cel laboratorium

Celem jest opanowanie systemu kontroli wersji Git, platformy GitHub oraz przygotowanie lokalnego środowiska programistycznego dla wybranego frameworka. Szczególny nacisk położono na poprawne dokumentowanie pracy przy użyciu Markdown, konfigurację bezpiecznego dostępu przez SSH oraz konfiguracja GitHub Actions.

2. Realizacja zadań

Zadanie 1: Konfiguracja środowiska i Markdown

•**Opis działań:** W ramach zadań skonfigurowałem swoją tożsamość w Git, aby moje commity były przypisane do właściwego imienia i adresu e-mail. Następnie wygenerowałem klucz SSH i dodałem go do konta GitHub, co pozwala na bezpieczne łączenie się z repozytoriami i przesyłanie zmian bez podawania hasła. Na końcu stworzyłem plik README.md, który zawiera nazwę projektu, krótki opis celu laboratorium, listę wymagań, przykładowy kod do uruchamiania serwera oraz link do oficjalnej dokumentacji Django.

•**Zastosowane komendy Git:**

•

•

•`$ git config --global user.name "Aleksandr Vasiljev"`

•`$ git config --global user.email "aleksvas2004@gmail.com"`

•`$ ssh-keygen -t ed25519 -C "aleksvas2004@gmail.com"`

- $ cat ~/.ssh/id_ed25519.pub
-
- $ ssh -T git@github.com
- **$ git clone git@github.com:aleksvas2004-rgb/ISI.git**

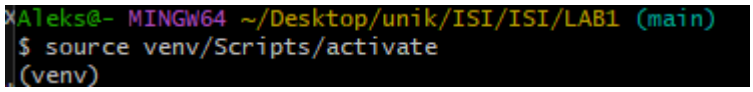
Zadanie 2: Inicjalizacja projektu i Git

- **Opis działań:** Na początku przygotowałem folder roboczy dla projektu i utworzyłem w nim wirtualne środowisko, aby wszystkie biblioteki były izolowane od systemowych instalacji Pythona. Następnie zainstalowałem framework Django i utworzyłem strukturę projektu w bieżącym katalogu. Po tym zainicjowałem repozytorium Git, przygotowałem plik requirements.txt z listą wszystkich bibliotek oraz stworzyłem plik .gitignore, aby wykluczyć z kontroli wersji środowisko wirtualne, pliki cache Pythona, lokalną bazę danych i pliki IDE. Na końcu sprawdziłem status repozytorium, dodałem wszystkie pliki do Git i wykonałem pierwszy commit zatytułowany „Initial commit: Django project structure".

- **Zastosowane komendy Git:**

- $ python -m venv venv
- **$ source venv/Scripts/activate**
- **$ pip install django**
- **$ django-admin startproject core .**
- **$ git init**
- **$ pip freeze > requirements.txt**
- **$ git status**
- **$ git add .**
- **$ git commit -m "Initial commit: Django project structure"**

- **Zrzuty ekranu / Fragmenty kodu:**



```
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ pip install django
Collecting django
  Downloading django-6.0.3-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref>=3.9.1 (from django)
  Downloading asgiref-3.11.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.5.0 (from django)
  Downloading sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Collecting tzdata (from django)
  Downloading tzdata-2025.3-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading django-6.0.3-py3-none-any.whl (8.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.4/8.4 MB 7.5 MB/s  0:00:01
Downloading asgiref-3.11.1-py3-none-any.whl (24 kB)
Downloading sqlparse-0.5.5-py3-none-any.whl (46 kB)
Downloading tzdata-2025.3-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.11.1 django-6.0.3 sqlparse-0.5.5 tzdata-2025.3

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ django-admin startproject core .
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git init
Initialized empty Git repository in C:/Users/Aleks/Desktop/unik/ISI/ISI/LAB1/.git/
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (master)
$ pip freeze > requirements.txt
(venv)
```

```
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        New Text Document.txt
        core/
        manage.py
        requirements.txt

nothing added to commit but untracked files present (use "git add" to track)
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (master)
$ git add .
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (master)
$ git commit -m "Initial commit: Django project structure"
[master (root-commit) 9ab8dc5] Initial commit: Django project structure
 9 files changed, 211 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 New Text Document.txt
 create mode 100644 core/__init__.py
 create mode 100644 core/asgi.py
 create mode 100644 core/settings.py
 create mode 100644 core/urls.py
 create mode 100644 core/wsgi.py
 create mode 100644 manage.py
 create mode 100644 requirements.txt
(venv)
```

Zadanie 3: Praca z gałęziami i podstawowa logika

•Opis działań: Utworzyłem nową gałąź funkcjonalną `feature/initial-setup` i przełączyłem się na nią, aby rozwijać projekt niezależnie od głównej gałęzi. W tej gałęzi stworzyłem aplikację `base` oraz zarejestrowałem ją w ustawieniach projektu, dodając ją do listy INSTALLED_APPS w pliku `core/settings.py`. Następnie dodałem wszystkie zmiany do Git i wykonałem commit z opisem dodania aplikacji. Po zakończeniu pracy na gałęzi przełączyłem się na główną gałąź `master`i scaliłem zmiany z gałęzi `feature/initial-setup`. Zaskoczyła mnie nazwa master głównej głęzi, której jak wiem już nie używa i szukałem jak ją zmienić, co w końcu udało się. W projekcie zachowałem co najmniej dwie gałęzie typu feature.

•Zastosowane komendy Git:

```
$ git checkout -b feature/initial-setup
•$ python manage.py startapp base
•$ git status
•$ git add .
•$ git commit -m "Add base application and register it in settings"
•$ git checkout master
•$ git merge feature/initial-setup
• git branch -m master main
•$ git branch
```

•Zrzuty ekranu / Fragmenty kodu:

```
leks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (master)
 git checkout -b feature/initial-setup
witched to a new branch 'feature/initial-setup'
(venv)
leks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (feature/initial-setup)
 python manage.py startapp base
(venv)
leks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (feature/initial-setup)
 git status
n branch feature/initial-setup
ntracked files:
  (use "git add <file>..." to include in what will be committed)
        base/

othing added to commit but untracked files present (use "git add" to track)
(venv)
leks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (feature/initial-setup)
 git add .
(venv)
leks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (feature/initial-setup)
 git commit -m "Add base application and register it in settings"
[feature/initial-setup 5b556aa] Add base application and register it in settings
 7 files changed, 17 insertions(+)
 create mode 100644 base/__init__.py
 create mode 100644 base/admin.py
 create mode 100644 base/apps.py
 create mode 100644 base/migrations/__init__.py
 create mode 100644 base/models.py
 create mode 100644 base/tests.py
 create mode 100644 base/views.py
(venv)
leks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (feature/initial-setup)
 git checkout main
rror: pathspec 'main' did not match any file(s) known to git
(venv)
leks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (feature/initial-setup)
 git checkout master
witched to branch 'master'
(venv)
```

```
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (master)
$ git merge feature/initial-setup
Updating 9ab8dc5..5b556aa
Fast-forward
 base/__init__.py            | 0
 base/admin.py               | 3 +++
 base/apps.py                | 5 +++++
 base/migrations/__init__.py | 0
 base/models.py              | 3 +++
 base/tests.py               | 3 +++
 base/views.py               | 3 +++
 7 files changed, 17 insertions(+)
 create mode 100644 base/__init__.py
 create mode 100644 base/admin.py
 create mode 100644 base/apps.py
 create mode 100644 base/migrations/__init__.py
 create mode 100644 base/models.py
 create mode 100644 base/tests.py
 create mode 100644 base/views.py
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (master)
$ git checkout main
error: pathspec 'main' did not match any file(s) known to git
(venv)
```

```
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (master)
$ git branch -m master main
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git merge feature/initial-setup
Already up to date.
(venv)
```

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'base',
]
```

Zadanie 4: Współpraca z GitHub

•Opis działań: Stworzyłem zdalne repozytorium na GitHub o nazwie ISI i połączyłem je z lokalnym projektem, aby móc przesyłać zmiany do serwera. Wykonałem pierwsze wypchnięcie całego kodu z lokalnego repozytorium. Następnie wysłałem również wszystkie gałęzie funkcjonalne typu feature, aby zachować ich historię na GitHubie. Dodatkowo w zakładce Issues utworzyłem kilka zadań do wykonania w kolejnych laboratoriach, takich jak integracja z zewnętrznym API czy dodanie autoryzacji, aby ułatwić planowanie dalszego rozwoju projektu. Nie zauważyłem jak stworzyłem konflikt. Stworzyłem .gitignore nie w tym miejscu co chciałem i scaliłem projekt, a potem usunąnem go w inne miejsce i zmieniłem zawartość, co spowodowało konflikt.

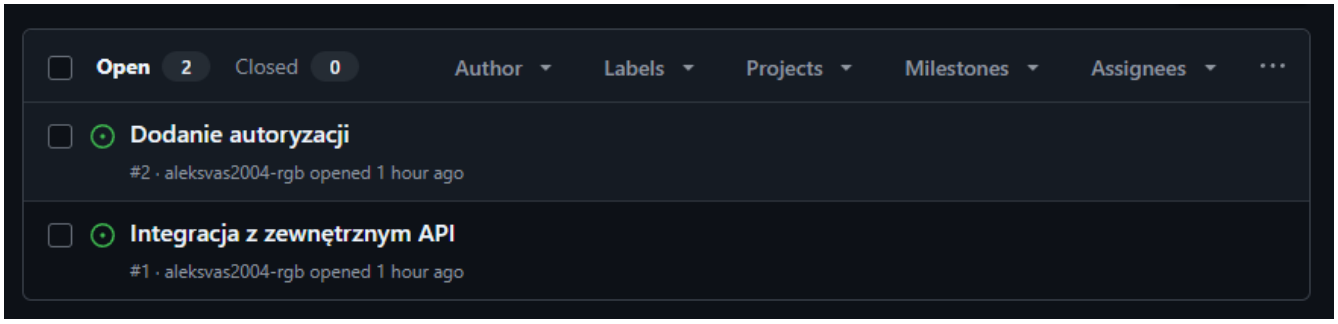
•Zastosowane komendy Git:

```
•git remote add origin git@github.com:aleksvas2004-rgb/ISI.git
•$ git push -u origin main
•$ git push origin feature/initial-setup
```

•Zrzuty ekranu / Fragmenty kodu:

```
$ git remote add origin git@github.com:aleksvas2004-rgb/ISI.git
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git push -u origin main
To github.com:aleksvas2004-rgb/ISI.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:aleksvas2004-rgb/ISI.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git status
On branch main
nothing to commit, working tree clean
(venv)
```

```
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git push -u origin main
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 12 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (20/20), 3.97 KiB | 1.99 MiB/s, done.
Total 20 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To github.com:aleksvas2004-rgb/ISI.git
   6876165..9d1548b  main -> main
branch 'main' set up to track 'origin/main'.
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git push origin feature/initial-setup
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 12 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (20/20), 3.80 KiB | 1.90 MiB/s, done.
Total 20 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
remote:
remote: Create a pull request for 'feature/initial-setup' on GitHub by visiting:
remote:      https://github.com/aleksvas2004-rgb/ISI/pull/new/feature/initial-setup
remote:
To github.com:aleksvas2004-rgb/ISI.git
 * [new branch]      feature/initial-setup -> feature/initial-setup
(venv)
```



Zadanie 5: Symulacja konfliktu

•Opis działań: Przeprowadziłem symulację konfliktu w repozytorium. Najpierw wprowadziłem zmianę w pliku README.md bezpośrednio na GitHubie, a równocześnie zmieniłem tę samą linię lokalnie w moim repozytorium. Podczas próby wypchnięcia zmian lokalnych Git odrzucił push, ponieważ wersja na serwerze była nowsza. Następnie pobrałem najnowsze zmiany z GitHub, co spowodowało konflikt w pliku README.md. Otworzyłem plik, rozwiązałem konflikt, wybierając odpowiednie fragmenty tekstu i usuwając znaczniki konfliktu, a następnie zatwierdziłem zmiany i wypchnąłem je na zdalne repozytorium.

•Zastosowane komendy Git:

```
•$ git add README.md
•$ git commit -m "Local change to README"
•$ git push origin main
•$ git pull origin main
•$ git commit -m "Fix merge conflict in README"
```

•Zrzuty ekranu / Fragmenty kodu:

```
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git add README.md
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git commit -m "Local change to README"
[main c661d3d] Local change to README
 1 file changed, 2 insertions(+), 1 deletion(-)
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git push origin main
To github.com:aleksvas2004-rgb/ISI.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:aleksvas2004-rgb/ISI.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git pull origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 939 bytes | 93.00 KiB/s, done.
From github.com:aleksvas2004-rgb/ISI
 * branch            main       -> FETCH_HEAD
   9d1548b..4b69066  main       -> origin/main
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main|MERGING)
$ git add README.md
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main|MERGING)
$ git commit -m "Fix merge conflict in README"
[main e2f89be] Fix merge conflict in README
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git pull origin main
From github.com:aleksvas2004-rgb/ISI
 * branch            main       -> FETCH_HEAD
Already up to date.
(venv)
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git push origin main
```

Zadanie 6: Współpraca z GitHub

- **Opis działań:Przygotowałem strukturę folderów w projekcie, tworząc katalog `.github/workflows`, w którym umieściłem plik `django_check.yml`. Plik ten definiuje workflow GitHub Actions, który przy każdym wypchnięciu kodu sprawdza poprawność składni Pythona i projektu Django przy użyciu narzędzia flake8. Po dodaniu i wypchnięciu pliku workflow na GitHub, w zakładce Actions**

obserwowałem jego działanie, czy poprawnie uruchamiany proces oznaczany był zielonym ptaszkiem. Następnie przetestowałem system, celowo wprowadzając błąd składniowy w jednym z plików projektu, co spowodowało zgłoszenie błędu przez GitHub Actions. Po weryfikacji naprawiłem błąd, aby workflow działał poprawnie. Jednocześnie stworzyłem nową gałąź workflow-test, którą za szybko skończyłem oraz na samym końcu sprawdziłem poprawność zadań za pomocą Listy kontrolnej.

•Zastosowane komendy Git:

```
•$ mkdir -p .github/workflows
```

•Zrzuty ekranu / Fragmenty kodu:

Event Status Branch Actor

new README.md
Django Syntax Check #8: Commit b4d206d pushed by aleksvas2004-rgb
main
Today at 3:56 PM
13s

merge conflict core/settings.py...
Django Syntax Check #7: Commit e71d3d9 pushed by aleksvas2004-rgb
main
Today at 3:49 PM
13s

fix branches
Django Syntax Check #6: Commit e4933ac pushed by aleksvas2004-rgb
main
Today at 3:46 PM
11s

workflow test
Django Syntax Check #5: Commit 3309400 pushed by aleksvas2004-rgb
feature/workflow-test
Today at 3:36 PM
10s

workflow fix
Django Syntax Check #4: Commit 0b94e6d pushed by aleksvas2004-rgb
main
Today at 3:34 PM
15s

workflow test nr2
Django Syntax Check #3: Commit 326a098 pushed by aleksvas2004-rgb
main
Today at 3:33 PM
12s

workflow test
Django Syntax Check #2: Commit 3309400 pushed by aleksvas2004-rgb
main
Today at 3:30 PM
13s

Add GitHub Actions workflow
Django Syntax Check #1: Commit 7a54ec0 pushed by aleksvas2004-rgb
main
Today at 3:23 PM
11s

3. Dokumentacja pracy z Gitem

•**Link do repozytorium:** https://github.com/aleksvas2004-rgb/ISI

•**Historia commitów:**

```
Aleks@- MINGW64 ~/Desktop/unik/ISI/ISI/LAB1 (main)
$ git log --oneline
b4d206d (HEAD -> main, origin/main, origin/HEAD) new README.md
e71d3d9 merge conflict core/settings.py 'base',
e4933ac fix branches
0b94e6d workflow fix
326a098 workflow test nr2
3309400 (origin/feature/workflow-test, feature/workflow-test) workflow test
7a54ec0 Add GitHub Actions workflow
e2f89be Fix merge conflict in README
c661d3d Local change to README
4b69066 Add subheading 'cos tu' under 'ISI' in README
9d1548b Add base application and register it in settings
ff3d3d7 Initial commit: Django project structure
5b556aa (origin/feature/initial-setup, feature/initial-setup) Add base application and register it in settings
9ab8dc5 Initial commit: Django project structure
6876165 Initial commit
(venv)
```

4. Wnioski

Podczas laboratorium opanowałem podstawową obsługę systemu kontroli wersji Git oraz platformy GitHub, w tym tworzenie i zarządzanie gałęziami, scalanie zmian oraz rozwiązywanie konfliktów. Nauczyłem się zarządzać zależnościami projektu. Szczególnie cenne było ćwiczenie pracy równoległej na gałęziach i symulacja konfliktu, dzięki czemu zrozumiałem, jak ważne jest śledzenie zmian. Konfiguracja kluczy SSH i GitHub Actions pozwoliła mi poznać bezpieczne i zautomatyzowane metody pracy z repozytorium. Nie duże trudności sprawiły początkowe konflikty między historią lokalną a zdalną, które nie zauwarzyłem oraz zmiana domyślnej nazwy gałęzi z `master` na `main`, ale dzięki nim zdobyłem praktyczne doświadczenie w rozwiązywaniu problemów Git.